Report for CDA 3201L Lab #5
Combinational Logic Design with Higher Level Devices

Name: Tuan Khang Phan **UID:** U30382645
Name: Huu Phat Nguyen **UID:** U46082380

I. Introduction

The purpose of this lab is to get familiar with higher level digital devices like decoders and multiplexers, and to see how hierarchical design makes building circuits easier. Instead of wiring up a bunch of individual logic gates, we can use ICs that already have multiple gates inside them, which saves time and reduces the number of chips on the breadboard. In this lab, we built a 4-to-1 multiplexer from basic gates (inverters, ANDs, ORs), then compared its output to a commercial 74LS153N multiplexer IC to make sure they behave the same way. We also wrote Verilog code for a 2-to-1 mux and used it to build a 4-to-1 mux hierarchically.

II. Methods and Materials

We used the following components and equipment:

- SN74LS04N inverter IC
- MM74HC11N 3-input AND gate IC
- SN74LS86AN 2-input XOR gate IC
- SN74LS153N dual 4-to-1 multiplexer IC
- Breadboard
- Jumper wires
- LED and resistor
- 5 V DC power supply

We started by writing out the truth tables for the 2-to-1 and 4-to-1 multiplexers and then derived the SOP expressions from those tables. Each product term in the 4-to-1 mux SOP has three inputs (two select lines and one data input). We used the MM74HC11N which provides 3-input AND gates directly, so each product term maps to exactly one gate without any cascading needed. No OR gate was required in this implementation.

We wired up the 74LS153N in parallel with our custom mux so both received the exact same inputs and select lines. To compare the two outputs, we connected them to a 74LS86N XOR gate followed by an inverter from the 74LS04N. When both outputs match, the XOR gives 0, and the inverter flips it to 1, lighting up the LED. If the outputs ever disagree, the LED turns off.

For testing, we set the data inputs to $I_0 = 1$, $I_1 = 0$, $I_2 = 1$, $I_3 = 0$ (the pattern 1010) by connecting them to VDD or GND accordingly. Then we tried all four select line combinations and observed the LED.

For the Verilog part, we wrote a simple 2-to-1 mux module and instantiated it three times to build a 4-to-1 mux. We wrote a testbench to go through all possible combinations and checked the output.

III. Results

1. Truth Table and Equations for Multiplexers

For a 2-to-1 multiplexer with data inputs A and B and select line S:

S	A	B	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

When $S = 0$, the output follows A regardless of B. When $S = 1$, the output follows B regardless of A. The SOP expression is: $Y = \overline{S} \cdot A + S \cdot B$

For a 4-to-1 multiplexer with data inputs I_0, I_1, I_2, I_3 and select lines S_1 and S_0:

S_1	S_0	Output
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

The SOP expression is:

$$Y = \overline{S_1} \cdot \overline{S_0} \cdot I_0 + \overline{S_1} \cdot S_0 \cdot I_1 + S_1 \cdot \overline{S_0} \cdot I_2 + S_1 \cdot S_0 \cdot I_3$$

2. Chip Count and Gate Analysis

To build the 4-to-1 multiplexer from basic gates, we needed:

- 2 inverters (from SN74LS04N) to generate $\overline{S_0}$ and $\overline{S_1}$
- 2 three-input AND gates (from MM74HC11N) for the product terms

For the comparator circuit, we used 1 XOR gate (from SN74LS86AN) and 1 additional inverter (from the same SN74LS04N). No OR gate was used in this implementation.

In total, the entire setup used: 1 x SN74LS04N (inverters), 1 x MM74HC11N (AND gates), 1 x SN74LS86AN (XOR comparator), plus the SN74LS153N as the reference mux.

3. Verilog Implementation

The design implements a 2-to-1 mux module and uses three instances of it to build the 4-to-1 mux hierarchically. The first two instances handle pairs of inputs using S_0 as the select, and the third picks between their outputs using S_1. The testbench cycles through all 64 possible combinations of the 6 inputs.

design.sv

```verilog
// Code your design here

module mux2(
    input  wire I0,
    input  wire I1,
    input  wire S,
    output wire Y
);
    assign Y = (~S & I0) | (S & I1);
endmodule


module mux4_discrete(
    input  wire I0, I1, I2, I3,
    input  wire S0, S1,
    output wire Q
);
    // SOP equation:
    // Q = ~S1~S0I0 + ~S1S0I1 + S1~S0I2 + S1S0I3
    assign Q =
        (~S1 & ~S0 & I0) |
        (~S1 &  S0 & I1) |
        ( S1 & ~S0 & I2) |
        ( S1 &  S0 & I3);
endmodule


module mux4_ref(
    input  wire I0, I1, I2, I3,
    input  wire S0, S1,
    output wire Y
);
    // Reference mux (like the 74LS153 output)
    // Same truth table as a 4:1 mux.
    assign Y = (S1 == 1'b0 && S0 == 1'b0) ? I0 :
               (S1 == 1'b0 && S0 == 1'b1) ? I1 :
               (S1 == 1'b1 && S0 == 1'b0) ? I2 :
                                            I3;
endmodule


module comparator_eq(
    input  wire A,
    input  wire B,
    output wire EQ
);
    // LED ON when equal
    assign EQ = ~(A ^ B);
endmodule
```

Figure 1: *Verilog design module for the 2-to-1 and 4-to-1 multiplexers.*

testbench.sv

```
// Code your testbench here
// or browse Examples


module tb_mux4_lab;
    reg I0, I1, I2, I3;
    reg S0, S1;

    wire Q_discrete;
    wire Y_ref;
    wire EQ;

    mux4_discrete U_DIS(.I0(I0), .I1(I1), .I2(I2), .I3(I3), .S0(S0), .S1(S1), .Q(Q_discrete));
    mux4_ref      U_REF(.I0(I0), .I1(I1), .I2(I2), .I3(I3), .S0(S0), .S1(S1), .Y(Y_ref));
    comparator_eq U_CMP(.A(Q_discrete), .B(Y_ref), .EQ(EQ));

    integer k;
    reg expected;

    initial begin
        // Waveform dump (GTKWave)
        $dumpfile("wave.vcd");
        $dumpvars(0, tb_mux4_lab);

        $display(" time  S1S0  I3I2I1I0  Q_discrete  Y_ref  EQ  expected  PASS?");
        $display("-------------------------------------------------------------");

        // Sweep ALL combinations:
        // Total bits = 6: I3 I2 I1 I0 S1 S0
        for (k = 0; k < 64; k = k + 1) begin
            {I3, I2, I1, I0, S1, S0} = k[5:0];
            #1;

            // Compute expected based on select
            case ({S1, S0})
                2'b00: expected = I0;
                2'b01: expected = I1;
                2'b10: expected = I2;
                2'b11: expected = I3;
            endcase

            $display("%4t    %b%b     %b%b%b%b        %b         %b     %b     %b       %s",
                     $time, S1, S0, I3, I2, I1, I0,
                     Q_discrete, Y_ref, EQ, expected,
                     ((Q_discrete===expected) && (Y_ref===expected) && (EQ===1'b1)) ? "PASS" : "FAIL");
        end

        $display("Done.");
        $finish;
    end
endmodule
```

Figure 2: *Testbench code used to verify the 4-to-1 multiplexer across all 64 input combinations.*

The simulation was run and all 64 test cases passed. A portion of the console output is shown below:

```
VCD info: dumpfile wave.vcd opened for output.
 time   S1S0   I3I2I1I0   Q_discrete   Y_ref   EQ   expected   PASS?
-----------------------------------------------------------------
   1     00      0000         0          0      1      0        PASS
   2     01      0000         0          0      1      0        PASS
   3     10      0000         0          0      1      0        PASS
   4     11      0000         0          0      1      0        PASS
   5     00      0001         1          1      1      1        PASS
   6     01      0001         0          0      1      0        PASS
   7     10      0001         0          0      1      0        PASS
   8     11      0001         0          0      1      0        PASS
  ...
  61     00      1111         1          1      1      1        PASS
  62     01      1111         1          1      1      1        PASS
  63     10      1111         1          1      1      1        PASS
  64     11      1111         1          1      1      1        PASS
Done.
testbench.sv:49: $finish called at 64 (1s)
```

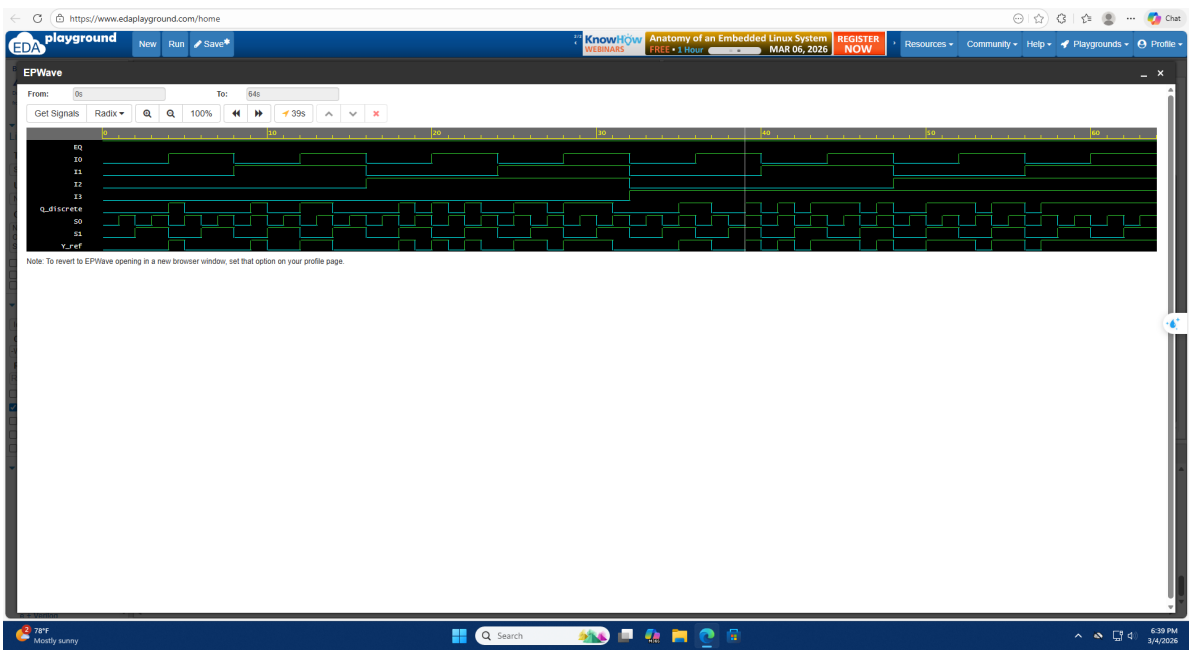

Figure 3: *Simulation waveform showing the 4-to-1 multiplexer output across all input combinations.*

4. Physical Circuit Testing

We set the data inputs to $I_0 = 1$, $I_1 = 0$, $I_2 = 1$, $I_3 = 0$ and tried each select combination. The results were:

S_1	S_0	Expected Output	LED
0	0	$I_0 = 1$	ON
0	1	$I_1 = 0$	OFF
1	0	$I_2 = 1$	ON
1	1	$I_3 = 0$	OFF

The LED turned on for $S_1S_0 = 00$ and $S_1S_0 = 10$, and stayed off for $S_1S_0 = 01$ and $S_1S_0 = 11$. This matches exactly what we expected since I_0 and I_2 are tied to HIGH while I_1 and I_3 are tied to LOW. Both our custom mux and the 74LS153N produced the same output for every combination we tested.

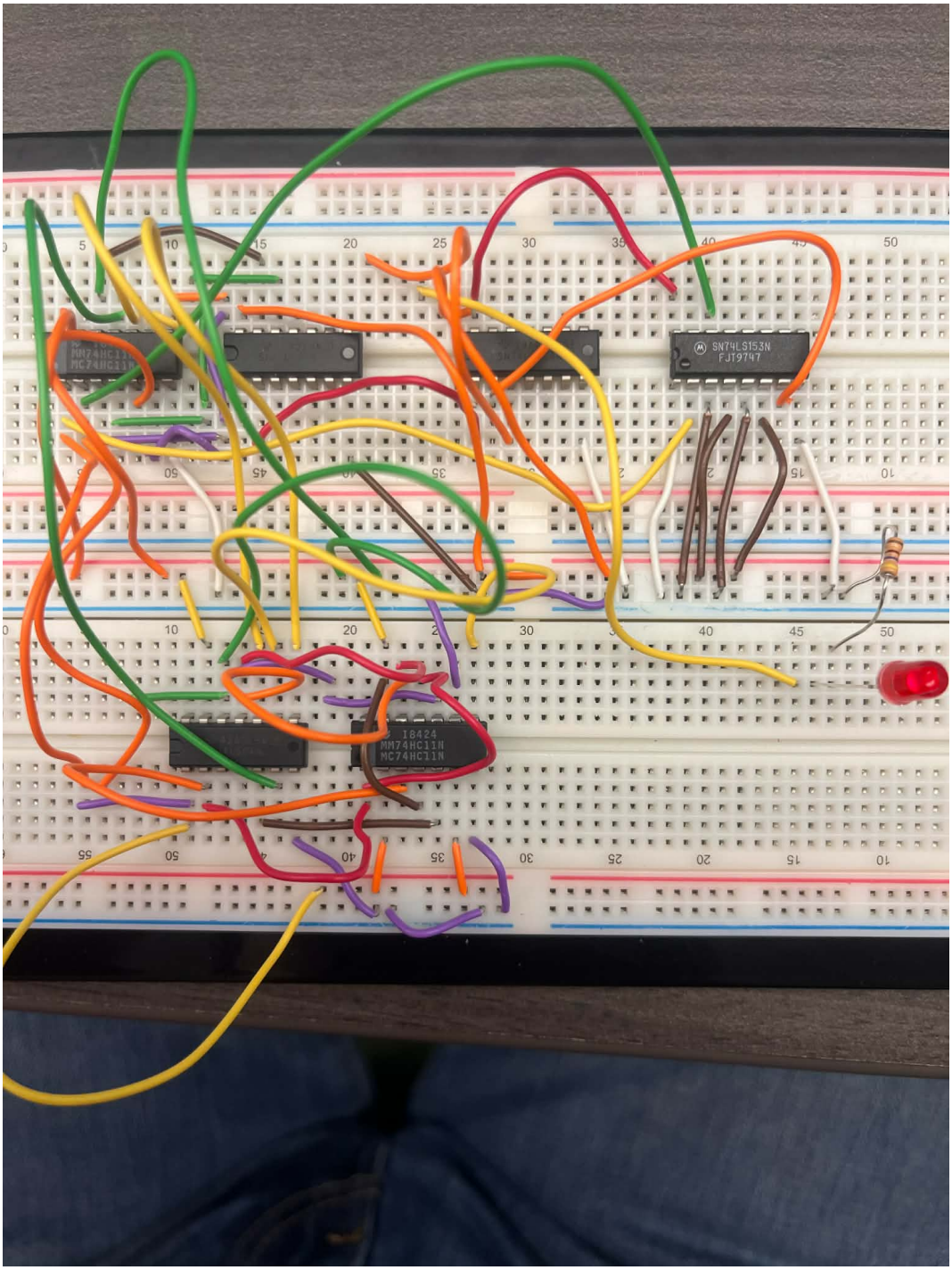

Figure 4: *Breadboard implementation of the 4-to-1 multiplexer circuit.*

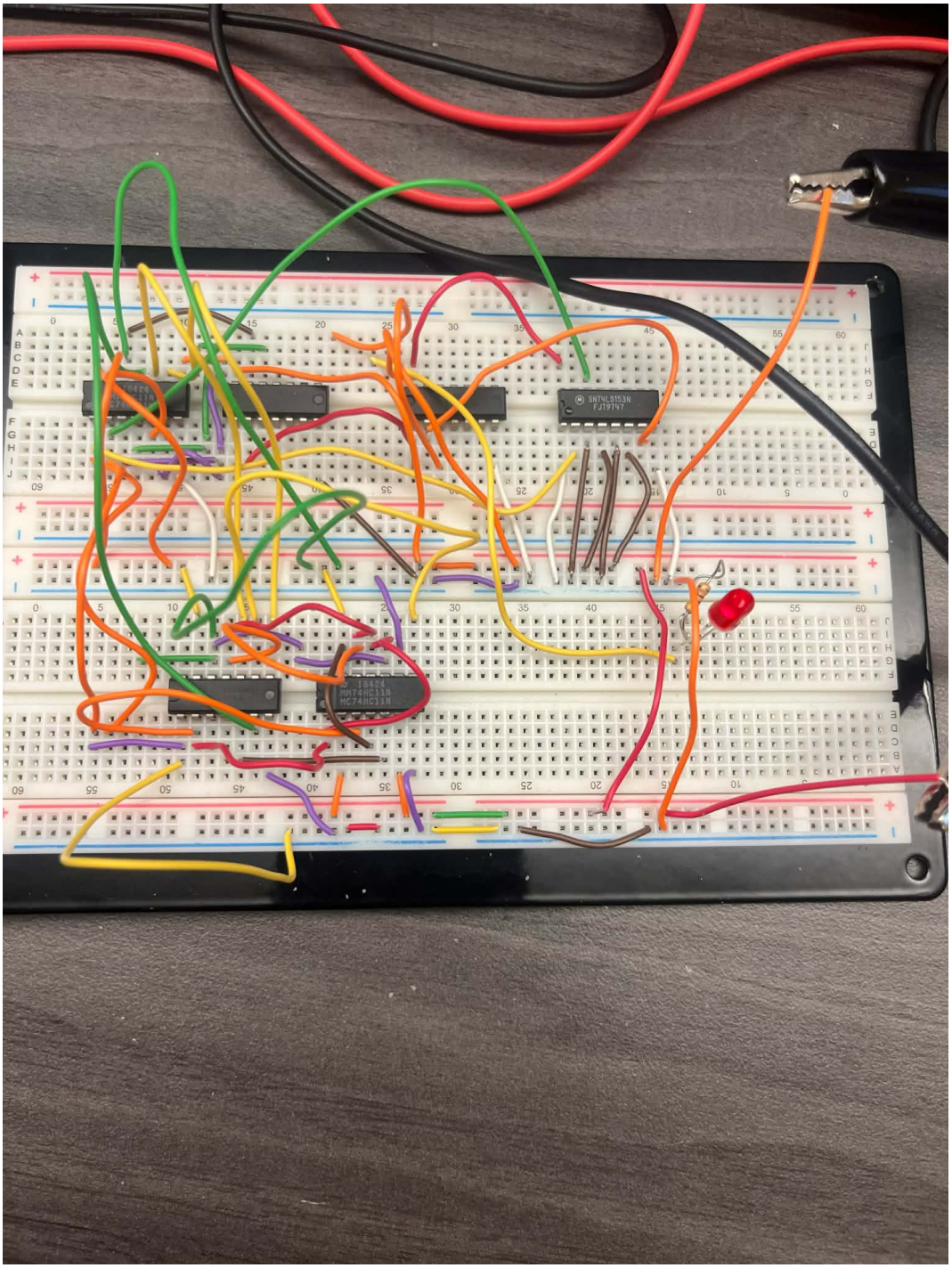

Figure 5: *Testing setup with both the custom mux and the 74LS153N connected in parallel.*

IV. Lab Assignment Questions

Question 1) Why is hierarchical design important, both when writing HDL code, and when implementing physical circuits? What are the benefits and potential drawbacks?

Hierarchical design lets us break a big circuit into smaller pieces that are easier to build and test

on their own. In Verilog, for example, we wrote a simple 2-to-1 mux and reused it three times to make a 4-to-1 mux. If one of those sub-modules has a bug, we can test it in isolation instead of debugging the whole thing at once. On the physical side, using ICs like the 74LS153N means we do not have to wire up dozens of individual gates, which saves breadboard space and reduces wiring mistakes.

The main benefit is reusability. Once a sub-module is tested and working, you can use it in many different designs without starting from scratch. The potential drawback is that a hierarchical design might not be as optimized as a flat one. For instance, building a 4-to-1 mux from three 2-to-1 muxes uses more gates than the minimum SOP implementation. There is also a learning curve when it comes to understanding how all the modules connect together, especially in larger designs.

Question 2) Explain how a decoder with active high/low outputs can be used to realize any SOP, or multiple SOP functions simultaneously. What are the limitations?

With an active HIGH decoder, each output corresponds to exactly one minterm. To implement any SOP function, we just OR together the decoder outputs for the minterms we need. For example, if $f(a,b,c) = \sum m(5,6,7)$, we connect outputs 5, 6, and 7 of a 3-to-8 decoder to an OR gate. We can even implement multiple functions from the same decoder by using different OR gates connected to different subsets of outputs.

With an active LOW decoder (where the selected output goes to 0 and all others stay at 1), we use NAND gates instead of OR gates to combine the desired minterms. This works because NANDing the active low outputs is logically equivalent to ORing active high outputs.

The main limitation is fan-out. Each decoder output can only drive a certain number of gate inputs before the signal degrades. If we try to connect one output to too many OR or NAND gates for different functions, it may exceed the drive strength of that output. Also, for functions with many variables, the decoder size grows exponentially (2^n outputs for n inputs), which becomes impractical for large n.

Question 3) How would you construct an 8-to-1 mux using 4-to-1 muxes?

We would use two 4-to-1 muxes and one 2-to-1 mux. The first 4-to-1 mux takes inputs I_0 through I_3 and uses S_0 and S_1 as its select lines. The second 4-to-1 mux takes inputs I_4 through I_7 with the same S_0 and S_1 select lines. Then the outputs of these two muxes go into a 2-to-1 mux that uses S_2 as its select line. When $S_2 = 0$, the output comes from the first group (inputs 0 through 3), and when $S_2 = 1$, it comes from the second group (inputs 4 through 7). This gives us a complete 8-to-1 selection with three select bits.

V. Discussion

The physical circuit worked as expected. With inputs set to 1010, the LED correctly turned on only when the selected input was HIGH ($S_1S_0 = 00$ selecting $I_0 = 1$ and $S_1S_0 = 10$ selecting $I_2 = 1$). The fact that both our custom mux and the 74LS153N agreed on every test case confirms that our gate level implementation of the SOP expression is correct.

One thing we noticed during the build is that the custom mux still takes up noticeably more space on the breadboard compared to just plugging in the SN74LS153N. Even with the MM74HC11N providing 3-input AND gates directly and eliminating the need for an OR gate, the wiring between inverters, AND gates, and the comparator adds up quickly. This is a good example of why higher

level ICs exist: they save board space, reduce wiring complexity, and are less prone to mistakes.

The Verilog implementation was straightforward once we had the 2-to-1 mux working. The hierarchical approach made it easy to scale up, and the testbench confirmed the output matched expectations for all 64 input combinations.

VI. Conclusion

In this lab, we built a 4-to-1 multiplexer from basic logic gates and verified it against a 74LS153N IC. The two circuits produced identical results for every input combination we tested. We also implemented the same multiplexer hierarchically in Verilog using 2-to-1 mux modules. This lab showed us how higher level devices and hierarchical design simplify both physical and HDL based circuit implementation, and why these approaches are preferred over building everything from individual gates.